```
import pandas as pd
import numpy as np
import warnings
import seaborn as sns
import matplotlib.pyplot as plt

warnings.filterwarnings('ignore')

from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from imblearn.over_sampling import SMOTE
```

Verideki Değişkenlerin Anketteki Sorulara Olan Karşılıkları

- *kalite:* Alınan ürünün kalitesi nasıl?
- *hatali_urun:* Hatalı ürün gönderme olasılığı nedir?
- *fiyat:* Ürünlerin fiyatı piyasa ortalamasına kıyasla nasıldır?
- hiz: Siparişler ne kadar hızlı ve zamanında teslim edilmektedir?
- *guvenilir_hizmet:* Sipariş teslimi ve sonrasında güvenilir hizmet sağlanıyor mu?
- *belge:* Ürünlerin garanti ve kalite belgesi bulunuyor mu?
- *talep:* Sorularınıza ve taleplerinize hızlı yanıt alabiliyor musunuz?
- *stok:* Tedarikçide ürün bulunma ve stok durumu nedir?
- *gecmis_donem:* Geçmiş dönem performansını nasıl değerlendirirsiniz?
- *kapasite:* Ürün gönderme kapasitesi nasıl?
- *tavsiye:* Tavsiye eder misiniz?

Veriye Genel Bakış

```
df = pd.read_excel('/content/veriler.xlsx')
df.head()
```

	tedarikci id	kalite	hatali_urun	fiyat	hiz	guvenilir_hizmet	belge	talep	stok	gecmis_donem	kapasite	tavsiye
0	1	iyi	orta	iyi	cok iyi	iyi	orta	iyi	cok iyi	iyi	cok iyi	Evet
1	2	orta	iyi	iyi	cok iyi	cok iyi	cok iyi	iyi	iyi	iyi	iyi	Evet
2	3	orta	orta	orta	iyi	orta	orta	iyi	orta	kotu	iyi	Evet
3	4	iyi	iyi	iyi	cok iyi	cok iyi	cok iyi	cok iyi	iyi	iyi	cok iyi	Evet
4	5	cok kotu	kotu	orta	orta	kotu	cok kotu	orta	iyi	kotu	orta	Hayir

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 169 entries, 0 to 168
Data columns (total 12 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   tedarikci id      169 non-null    int64
 1   kalite            169 non-null    object
 2   hatali_urun       169 non-null    object
 3   fiyat             169 non-null    object
 4   hiz               169 non-null    object
 5   guvenilir_hizmet  169 non-null    object
 6   belge             169 non-null    object
 7   talep             169 non-null    object
 8   stok              169 non-null    object
 9   gecmis_donem      169 non-null    object
 10  kapasite          169 non-null    object
 11  tavsiye           169 non-null    object
dtypes: int64(1), object(11)
memory usage: 16.0+ KB
```

```
df.isnull().sum()
```

	0
tedarikci id	0
kalite	0
hatali_urun	0
fiyat	0
hiz	0
guvenilir_hizmet	0
belge	0
talep	0
stok	0
gecmis_donem	0
kapasite	0
tavsiye	0

dtype: int64

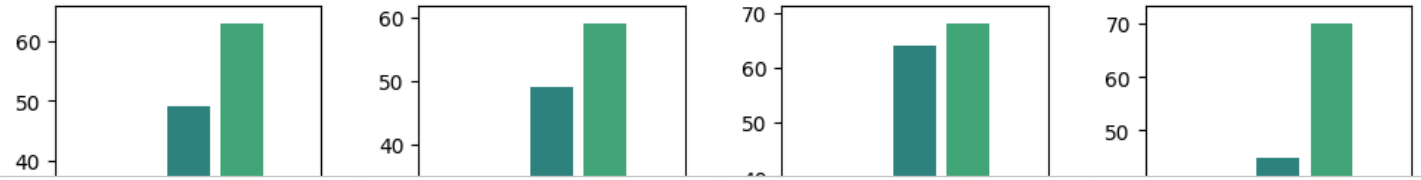
```
df['tavsiye'].value_counts()
```

tavsiye	count
Evet	122
Hayir	47

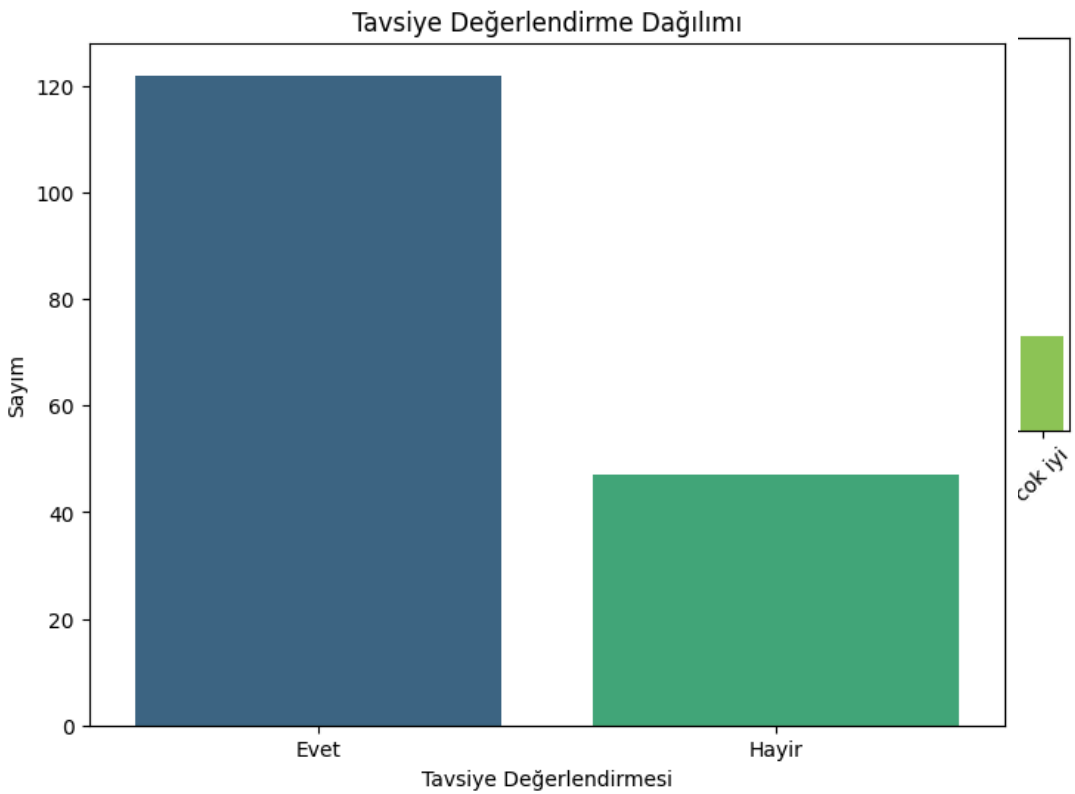
dtype: int64

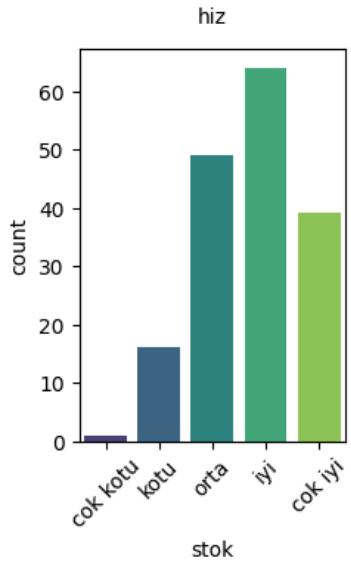
Değişkenlerin Dağılımları

```
siralama = ["cok kotu", "kotu", "orta", "iyi", "cok iyi"]
plt.figure(figsize=(10, 11))
for i, col in enumerate(df.columns[1:-1]):
    plt.subplot(3, 4, i+1)
    sns.countplot(x=col, data=df, palette="viridis", order=siralama)
    plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(1, 1, figsize=(8, 6))

sns.countplot(data=df, x='tavsiye', palette="viridis", ax=axes, )

axes.set_xlabel('Tavsiye Değerlendirmesi')
axes.set_ylabel('Sayım')
axes.set_title('Tavsiye Değerlendirme Dağılımı')

plt.show()
```






```
df.drop(columns=["tedarikci id"], inplace=True)
```

```
mapping = {"cok kotu": 0, "kotu": 1, "orta": 2, "iyi": 3, "cok iyi": 4}
binary_mapping = {"Evet": 1, "Hayir": 0}

for col in df.columns:
    if df[col].dtype == 'object' and col != "tavsiye":
        df[col] = df[col].map(mapping)

df["tavsiye"] = df["tavsiye"].map(binary_mapping)

X = df.drop("tavsiye", axis=1)
y = df["tavsiye"]
smote = SMOTE(random_state=42)
X_resampled, y_resampled = smote.fit_resample(X, y)

df_smote = pd.concat([pd.DataFrame(X_resampled, columns=X.columns), pd.DataFrame(y_resampled, columns=['tavsiye'])], axis=1)

df_smote["tavsiye"] = df_smote["tavsiye"].map({1: "Evet", 0: "Hayir"})
for col in X.columns:
    df_smote[col] = df_smote[col].map({v: k for k, v in mapping.items()})

df_smote.to_csv("dengelenmis_veri.csv", index=False)

print(df_smote["tavsiye"].value_counts())
```

```
tavsiye
Evet     122
Hayir    122
Name: count, dtype: int64
```

```
df = pd.read_csv("/content/dengelenmis_veri.csv")
```

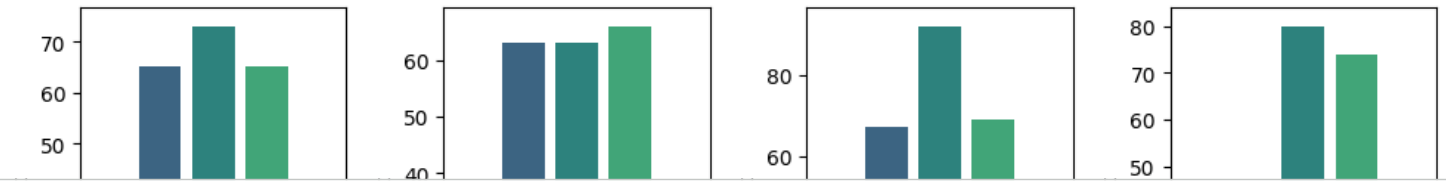
```
df.head()
```

	kalite	hatali_urun	fiyat	hiz	guvenilir_hizmet	belge	talep	stok	gecmis_donem	kapasite	tavsiye
0	iyi	orta	iyi	cok iyi	iyi	orta	iyi	cok iyi	iyi	cok iyi	Evet
1	orta	iyi	iyi	cok iyi	cok iyi	cok iyi	iyi	iyi	iyi	iyi	Evet
2	orta	orta	orta	iyi	orta	orta	iyi	orta	kotu	iyi	Evet
3	iyi	iyi	iyi	cok iyi	cok iyi	cok iyi	cok iyi	iyi	iyi	cok iyi	Evet
4	cok kotu	kotu	orta	orta	kotu	cok kotu	orta	iyi	kotu	orta	Hayir

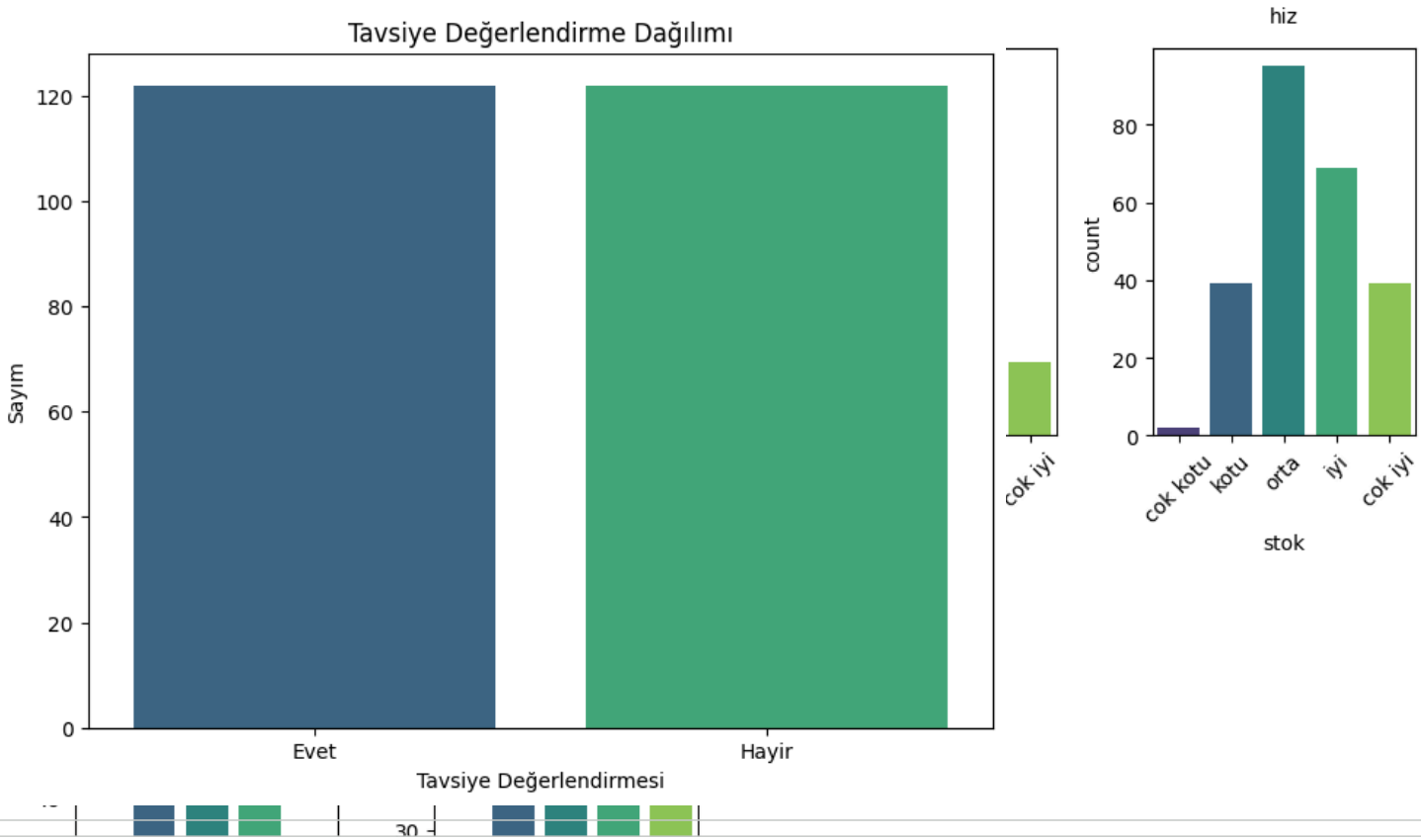
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 11 columns):
 #   Column            Non-Null Count  Dtype
---  ------            --------------  -----
 0   kalite            244 non-null    object
 1   hatali_urun       244 non-null    object
 2   fiyat             244 non-null    object
 3   hiz               244 non-null    object
 4   guvenilir_hizmet  244 non-null    object
 5   belge             244 non-null    object
 6   talep             244 non-null    object
 7   stok              244 non-null    object
 8   gecmis_donem      244 non-null    object
 9   kapasite          244 non-null    object
 10  tavsiye           244 non-null    object
dtypes: object(11)
memory usage: 21.1+ KB
```

```
siralama = ["cok kotu", "kotu", "orta", "iyi", "cok iyi"]
plt.figure(figsize=(10, 11))
for i, col in enumerate(df.columns[0:-1]):
    plt.subplot(3, 4, i+1)
    sns.countplot(x=col, data=df, palette="viridis", order=siralama)
    plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



```
fig, axes = plt.subplots(1, 1, figsize=(8, 6))

sns.countplot(data=df, x='tavsiye', palette="viridis", ax=axes, )

axes.set_xlabel('Tavsiye Değerlendirmesi')
axes.set_ylabel('Sayım')
axes.set_title('Tavsiye Değerlendirme Dağılımı')

plt.show()
```




```
le = LabelEncoder()
```

```
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
```

```
df.head()
```

	kalite	hatali_urun	fiyat	hiz	guvenilir_hizmet	belge	talep	stok	gecmis_donem	kapasite	tavsiye
0	2	4	2	0	2	4	2	0	2	0	0
1	4	2	2	0	0	0	2	2	2	2	0
2	4	4	4	2	4	4	2	4	3	2	0
3	2	2	2	0	0	0	0	2	2	0	0
4	1	3	4	4	3	1	4	2	3	4	1

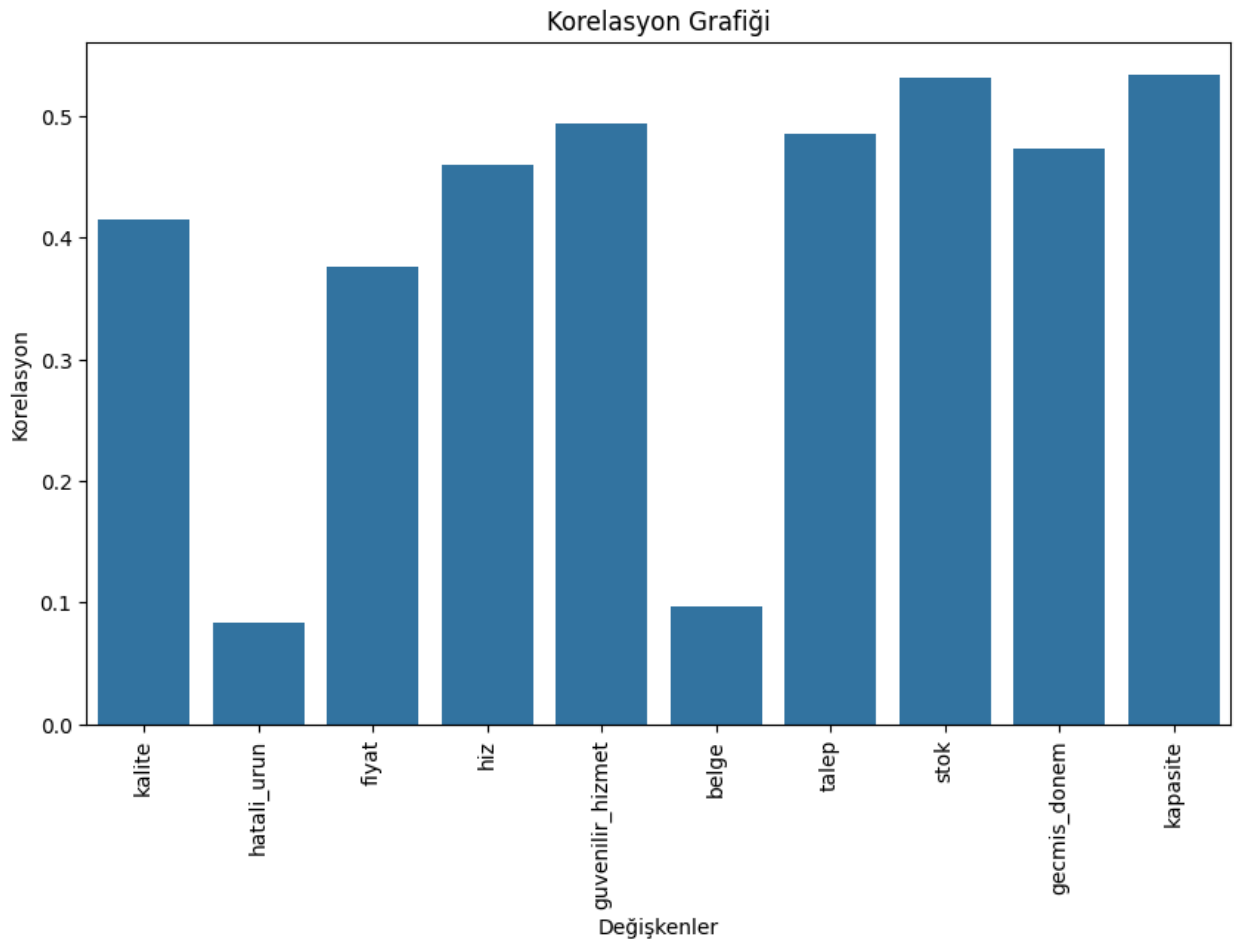
Hedef Değişkene Göre Korelasyonların Hesaplanması

```
correlations = df.corr()['tavsiye'].drop(['tavsiye'])

print(correlations)
```

```
kalite              0.414341
hatali_urun         0.084007
fiyat               0.375476
hiz                 0.459336
guvenilir_hizmet    0.493164
belge               0.096768
talep               0.485672
stok                0.531413
gecmis_donem        0.473052
kapasite            0.533979
Name: tavsiye, dtype: float64
```

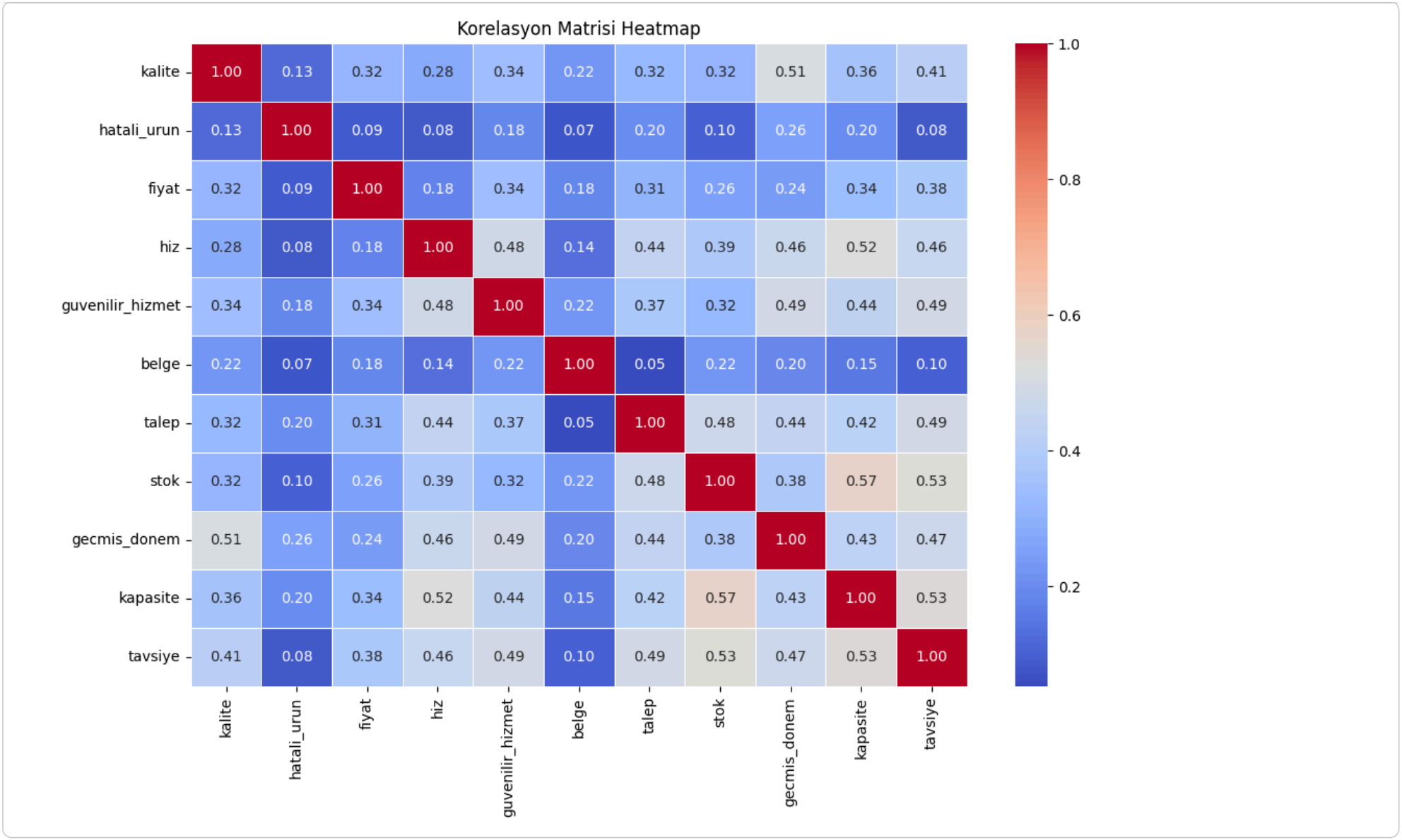
Korelasyonu Grafik ile Görselleştirme

```
plt.figure(figsize=(10, 6))
sns.barplot(x=correlations.index, y=correlations.values)
plt.xlabel('Değişkenler')
plt.ylabel('Korelasyon')
plt.title('Korelasyon Grafiği')
plt.xticks(rotation=90)

plt.show()
```



Korelasyon Matrisini Heatmap Olarak Görselleştirme

```
corr_matrix = df.corr()

plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5)
plt.title('Korelasyon Matrisi Heatmap')

plt.show()
```



Modelin Oluşturulması

Veri Yükleme

```
data = pd.read_csv("/content/dengelenmis_veri.csv")
```

Farklı Modellerin CV Skoruna Bakılması

```
X = data.drop(columns=['tavsiye'])
y = data['tavsiye']

y = y.astype('category')

for col in X.columns:
    if X[col].dtype == 'object':
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col])

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Support Vector Machine": SVC()
}

cv_scores = {}
classification_reports = {}

for model_name, model in models.items():
    try:
        cv_results = cross_validate(model, X, y, cv=5,
                                    scoring=['accuracy', 'precision_macro', 'recall_macro', 'f1_macro'],
                                    return_train_score=False)

        cv_scores[model_name] = np.mean(cv_results['test_accuracy'])

        classification_reports[model_name] = {
            "Precision": np.mean(cv_results['test_precision_macro']),
            "Recall": np.mean(cv_results['test_recall_macro']),
            "F1-score": np.mean(cv_results['test_f1_macro']),
        }
    except Exception as e:
        print(f"Error with model {model_name}: {e}")
        continue

cv_scores_df = pd.DataFrame(list(cv_scores.items()), columns=["Model", "CV Accuracy"])

classification_data = []
```

```python
for model_name, metrics in classification_reports.items():
    model_report = {
        "Model": model_name,
        "Precision": metrics["Precision"],
        "Recall": metrics["Recall"],
        "F1-score": metrics["F1-score"],
    }
    classification_data.append(model_report)

classification_df = pd.DataFrame(classification_data)

final_results = pd.merge(cv_scores_df, classification_df, on="Model")

final_results = final_results.round(2)

final_results.to_csv("model_performance_results.csv", index=False)

print("Results saved to 'model_performance_results.csv'")
```

```
Error with model Logistic Regression: name 'cross_validate' is not defined
Error with model Decision Tree: name 'cross_validate' is not defined
Error with model Random Forest: name 'cross_validate' is not defined
Error with model K-Nearest Neighbors: name 'cross_validate' is not defined
Error with model Support Vector Machine: name 'cross_validate' is not defined
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-20-67832ef15b23> in <cell line: 0>()
     52 classification_df = pd.DataFrame(classification_data)
     53
---> 54 final_results = pd.merge(cv_scores_df, classification_df, on="Model")
     55
     56 final_results = final_results.round(2)

/usr/local/lib/python3.11/dist-packages/pandas/core/reshape/merge.py in merge(left, right, how, on, left_on, right_on, left_index, right_index, sort, suffixes, copy,
indicator, validate)
    168         )
    169     else:
--> 170         op = _MergeOperation(
    171             left_df,
    172             right_df,

                              2 frames
/usr/local/lib/python3.11/dist-packages/pandas/core/generic.py in _get_label_or_level_values(self, key, axis)
   1909             values = self.axes[axis].get_level_values(key)._values
```

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import cross_validate
```